

AISG 3.0 포팅 90일 작업 계획



전체 일정 한눈에

Week 01-02	Phase 1 - 기존 C++ 소스 분석 + 환경 셋업
Week 03-04	Phase 2 - AISG 2.0 동작 검증 + 단위 테스트 베이스
Week 05-08	Phase 3 - AISG 3.0 변경점 4개 구현 (핵심)
Week 09-10	Phase 4 - 통합 테스트 (자체 시뮬레이션)
Week 11-12	Phase 5 - 현장 연동 테스트 (용인)
Week 13	Phase 6 - 디버깅 / 안정화 / 인계

각 Phase별 마일스톤 + 산출물 + 위험은 아래 상세.

Phase 1 — 기존 C++ 소스 분석 + 환경 셋업 (Week 1~2)

목표

- 기존 AISG 2.0 C++ 소스 코드의 **전체 구조 파악**
- 빌드 환경 / 컴파일러 / 타겟 MCU 확정
- AISG 2.0 동작 흐름 도식화

작업

- D-Day:** 계약 체결 + NDA + 소스 코드 인수
- Week 1:**
 - 디렉토리 구조 / 빌드 시스템 (Makefile? CMake? Keil?) 파악
 - 진입점 (main, init) 식별 + 콜백 흐름
 - 레이어 분리: PHY / DLL / Application 모듈 위치
 - state machine 식별 (명시적 또는 암묵적)
- Week 2:**
 - 타겟 MCU 환경에서 기존 코드 컴파일·실행
 - 시뮬레이션 환경 구축 (PC + UART 시리얼)
 - 기존 동작 검증 (회귀 테스트 베이스)

산출물

- 01_existing_code_analysis.md** — 기존 코드 구조 분석서
- 02_build_environment_setup.md** — 빌드·디버그 환경 셋업
- 03_aisg_2_0_state_diagram.md** — 동작 상태도 (PNG + 설명)
- 기존 코드 빌드·실행 동영상 (1~2분)

마일스톤

- ✅ 기존 코드 100% 컴파일 통과
- ✅ 기존 코드 기본 동작 확인 (송신·수신 1건씩)
- ✅ 클라이언트와 1차 진척 보고 미팅 (Week 2 종료 시점)

위험 / 완화

- **위험:** 기존 코드 품질 낮음 / 주석 부재 → 분석 시간 ↑
- **완화:** Phase 1에 2주 충분히 배정. 만약 Week 2 종료까지 분석 못 끝나면 클라이언트와 즉시 협의 (일정 조정 / Scope 축소)

Phase 2 — AISG 2.0 동작 검증 + 단위 테스트 베이스 (Week 3~4)

목표

- 기존 AISG 2.0 동작이 정확히 어떤 메시지·명령을 처리하는지 명확화
- 회귀 테스트 베이스 라인 구축 (3.0 포팅 후 깨지지 않게 확인용)

작업

1. Week 3:

- AISG 2.0 명령 코드 / 응답 코드 매핑표 작성
- 각 명령에 대한 단위 테스트 케이스 작성 (Python 또는 C로 시뮬레이션)
- HDLC 프레임 파서 / 생성기 동작 검증
- CRC-CCITT 16 계산 검증

2. Week 4:

- 클라이언트 측 실 ALD 장비 (있다면) 또는 시뮬레이션 모델 으로 송수신 확인
- 통신 로그 캡처 + 기존 동작과 일치 확인
- 회귀 테스트 자동화 스크립트 작성

산출물

- 04_aisg_2_0_command_map.md — 명령·응답 코드 매핑
- 05_regression_test_suite.md — 회귀 테스트 케이스 50+
- tests/ — Python pytest 자동화 스크립트
- AISG 2.0 동작 검증 동영상 (3~5분)

마일스톤

- ☒ AISG 2.0 모든 명령 회귀 테스트 PASS
- ☒ 클라이언트와 2차 진척 보고 미팅 (Week 4 종료)

위험 / 완화

- **위험:** 실 ALD 장비 부재 → 시뮬레이션만으로 검증 한계
- **완화:** 사무실에 자체 모의 ALD (RS-485 모뎀 + 응답 시뮬레이터) 구축. 또는 클라이언트 현장 1회 방문해서 캡처.

Phase 3 — AISG 3.0 변경점 4개 구현 (Week 5~8) ★ 핵심

목표

- AISG 3.0 신규 4가지 기능 모두 구현 완료
- 각 기능별 단위 테스트 통과

Week 5 — Device Discovery

구현:

- 버스 broadcast 명령 (0xFF 또는 AISG 3.0 표준 명령) 송신
- 슬레이브 응답 타임아웃 (200~500 ms)
- 디바이스 목록 자료 구조 (struct array + dynamic)
- 새 디바이스 hot-plug 감지 (주기적 재스캔)

테스트:

- 1~10대 디바이스 시뮬레이션 환경에서 100% 감지

Week 6 — Multi-Primary

구현:

- 컨트롤러별 세션 상태 분리 (struct array indexed by controller_id)
- 동시 접근 방지 (mutex / 세마포어)
- 트랜잭션 ID 기반 응답 매칭
- 우선순위·타임아웃 정책

테스트:

- 2~4개 컨트롤러 동시 접근 시나리오에서 충돌 0건

Week 7 — Connection Mapping

구현:

- Ping 패킷 송신기 (RF 채널별)
- 응답 수집 → RF 채널 ↔ ALD 매핑 테이블
- 매핑 테이블 영속화 (Flash 또는 EEPROM 저장)
- 매핑 변경 감지 (재스캔)

테스트:

- 매핑 테이블 정합성 검증 + 케이블 swap 시나리오

Week 8 — Ping Packet + 통합 단위 테스트

구현:

- Ping 송신·수신 모듈 (모든 RF 채널)
- 장애 감지 로직 (응답 없는 채널 식별)
- 로그·알람 처리

테스트:

- 4개 기능 통합 단위 테스트 PASS
- 회귀 테스트 (Phase 2 베이스) 깨지지 않음 확인

산출물 (Phase 3 종합)

- 기능별 4개 모듈 소스 (clean code + 주석)
- [06_aisg_3_0_implementation.md](#) — 각 기능 구현 보고
- [07_aisg_3_0_unit_tests.md](#) — 단위 테스트 결과
- Phase 3 종료 시 데모 동영상 (5~10분)

마일스톤

- ☒ Week 5 종료: Device Discovery 데모
- ☒ Week 6 종료: Multi-Primary 데모
- ☒ Week 7 종료: Connection Mapping 데모
- ☒ Week 8 종료: Ping + 통합 단위 테스트 → 클라이언트 3차 미팅

위험 / 완화

- **위험:** Multi-Primary 동시성 버그 (race condition / deadlock)
- **완화:** BLE Mesh 3,800대 시스템의 검증된 mutex 패턴 재사용. valgrind 또는 ThreadSanitizer 활용 가능 (타깃 환경 따라).

Phase 4 — 통합 테스트 (자체 시뮬레이션) (Week 9~10)

목표

- AISG 3.0 4개 기능을 자체 시뮬레이션 환경에서 안정성 검증
- 부하·내구성·에지 케이스 테스트

작업

1. Week 9:

- 1~10대 ALD 시뮬레이터 환경 구축 (PC + 자체 모델)
- 부하 테스트 — 동시 명령 100건 / 1시간 연속 동작
- 에지 케이스 — 응답 없음 / 충돌 / 케이블 끊김

2. Week 10:

- 메모리 누수 검사
- 24시간 연속 동작 검증
- 모든 회귀 테스트 PASS
- 코드 리뷰 (자체)

산출물

- [08_integration_test_report.md](#) — 통합 테스트 결과 + 발견 이슈
- 메모리 사용량 측정표
- 24시간 안정성 로그

마일스톤

- ☒ Week 10 종료: 24시간 연속 동작 PASS + 클라이언트 4차 미팅
- ☒ Phase 5 (현장 테스트) 진입 결정

위험 / 완화

- **위험:** 자체 시뮬레이터의 현실성 부족 — 현장과 차이 발생 가능
- **완화:** Phase 5에서 1주차에 빠른 현장 1차 방문으로 차이 확인

Phase 5 — 현장 연동 테스트 (용인 기흥구) (Week 11~12)

목표

- 실 ALD 장비 + 실 RF 케이블 환경에서 AISG 3.0 동작 검증
- 현장 변수 (RF 노이즈 / 케이블 손실 / 환경 온도 등) 대응

Week 11 — 1차 현장

작업:

- 현장 셋업 + 통신 수립 확인
- Device Discovery 실증 (실 ALD 1~3대 인식)
- Multi-Primary 실증 (가능하다면)
- 발견 이슈 즉시 사무실로 가져가 수정 (용인 동일 지역 → 당일 가능)

Week 12 — 2차 / 3차 현장 (필요 시)

작업:

- Connection Mapping + Ping 실증
- 부하 테스트 (실 환경)
- 내구성 (수 시간 연속 동작)
- 인증 적합성 사전 점검 (필요 시 AISG 인증 절차 협의)

산출물

- 09_field_test_report.md — 현장 테스트 결과 + 발견 이슈 + 해결
- 현장 사진·동영상
- 통신 로그 캡처

마일스톤

- ☒ Week 11 종료: 1차 현장 통신 수립 + 클라이언트 5차 미팅
- ☒ Week 12 종료: 모든 4개 기능 현장 실증

위험 / 완화

- **위험:** 현장 환경에서만 발생하는 RF 노이즈 / 케이블 손실 이슈
- **완화:** 임호균 협업자 동행 — HW 측 즉시 진단. 자체 사무실 동일 지역 → 디버깅 사이클 짧음

Phase 6 — 디버깅 / 안정화 / 인계 (Week 13)

목표

- 발견된 모든 이슈 해결 + 문서 정리 + 클라이언트 인계

작업

- 잔여 버그 수정
- 코드 리팩토링 (가독성·유지보수성)
- 사용자 매뉴얼 작성
- 인계 미팅 (전체 시연 + 코드 walk-through)
- 보증 기간 정책 합의 (3~6개월 권장)

산출물

- 최종 소스 코드 (clean + 주석 + README)
- `10_user_manual.md` — 클라이언트가 향후 유지보수에 쓸 매뉴얼
- `11_known_issues.md` — 알려진 이슈 + 해결 권장
- 최종 인계 동영상 (10~15분 전체 시연)
- 보증 기간 합의서

마일스톤

-  Week 13 종료: 최종 인계 미팅 + 검수 통과



마일스톤 요약

시점	마일스톤	클라이언트 미팅
Week 2 종료	기존 코드 분석 완료	 1차
Week 4 종료	AISG 2.0 회귀 테스트 PASS	 2차
Week 8 종료	AISG 3.0 4개 기능 단위 테스트 PASS	 3차
Week 10 종료	통합 테스트 PASS (24h 연속 동작)	 4차
Week 11 종료	1차 현장 통신 수립	 5차
Week 13 종료	최종 인계	 6차 (최종)

→ **6회 진척 미팅** = 클라이언트 입장 안심. 진행 가시성 확보.



도구 / 환경

영역	사용 도구
컴파일러	(클라이언트 기존 환경에 맞춰 — GCC ARM / Keil / IAR)
빌드	(기존 — Makefile / CMake / Keil project)
디버그	J-Link / ST-Link (보유)
통신 분석	UART/RS-485 로직 분석기 (보유)
시뮬레이션	자체 PC 모뎀 (자체 제작) + Python pytest
형상 관리	Git (private repository) + NDA 준수
문서	Markdown + Mermaid 다이어그램

 일정 위험 시나리오 + 완화

시나리오	영향	완화
기존 코드 분석 1주 추가 소요	-1주 버퍼 소진	만약 -2주 이상이면 클라이언트와 즉시 협의 (Scope 축소 또는 일정 연장)
실 ALD 장비 부재	Phase 5 단축	자체 시뮬레이터로 95% 검증 + 현장은 통합 확인 위주
Multi-Primary 동시성 버그	-1주	mutex 재설계 + ThreadSanitizer 활용
인증 절차 필요 발견	별도 견적	본 90일 Scope 밖 → 별도 협의
현장 RF 환경 문제	임호균 동행	HW + SW 동시 디버깅 (사무실 동일 지역 활용)

메타

항목	값
작성일	2026-05-12
일정 가정	D-day = 계약 체결 후 즉시 / 90일 = 약 13주
마일스톤 미팅 횟수	6회 (Phase별 1회)
클라이언트 입장 가시성	매 2주마다 진척 확인 가능
작업 계획 검증	미팅에서 클라이언트와 합의 → 계약서에 명시